

Afillar Insurance APIs

Developer Guide

Code, configuration, testing and maintenance reference

Version 1.1

02-Oct-2026

Prepared for the Afillar API development and maintenance team

Table of Contents

Table of Contents	2
1. Introduction	3
1.1 Purpose of this Guide	3
1.2 What the Solution Is	3
1.3 Glossary	3
2. Architecture	4
2.1 Layers	4
2.2 Design Rules	4
2.3 Request Lifecycle	4
3. Source Layout and Tooling	6
3.1 Workspace Layout	6
3.2 The imo Tool	6
3.3 Groovy Rules of This Runtime	6
3.4 Deployment Order of the Code	6
4. Code Reference	8
4.1 Asset Inventory	8
4.2 Common Functions	9
4.3 APIs	14
5. API Walkthroughs	17
6. Data and Configuration Reference	18
6.1 Tables	18
6.2 Configuration Keys (UP_ApiConfig)	19
6.3 Field Rules (UP_FieldRule)	21
6.4 Security Baselines (UPSecurity)	22
7. Platform Configuration (Products)	23
7.1 What Exists on the Platform	23
7.2 Data Dictionary Bindings	23
7.3 How the Products Were Built	23
8. Business Algorithms	24
8.1 Premium Mode, Term and Age	24
8.2 Cancellation (Checker / Approver)	24
8.3 Platform Quirks That Shaped the Code	25
9. Errors, Logging and Tracing	26
10. How-To Recipes	27
11. Test Tooling	29
12. Test Procedure for a Change	31
13. Environments and Deployment	32
14. Known Limitations and Backlog	33
14.1 Troubleshooting	33
14.2 First-Day Checklist for a New Developer	33
Appendix A. File Inventory	34
Appendix B. Table Definitions	35

1. Introduction

1.1 Purpose of this Guide

This guide is for the developers who will maintain and extend the Afillar Insurance Company insurance APIs after hand-over. It explains how the solution is built, where every piece of logic and data lives, how a request travels through the code, which platform services are called and why, which platform quirks shaped the design, how to change or add things safely, how to test, and how to deploy. The reference sections (4, 6, 11, 12 and the appendices) are generated from the real source files, tables and scripts, so they match what is deployed in the MC tenant at the time of writing.

1.2 What the Solution Is

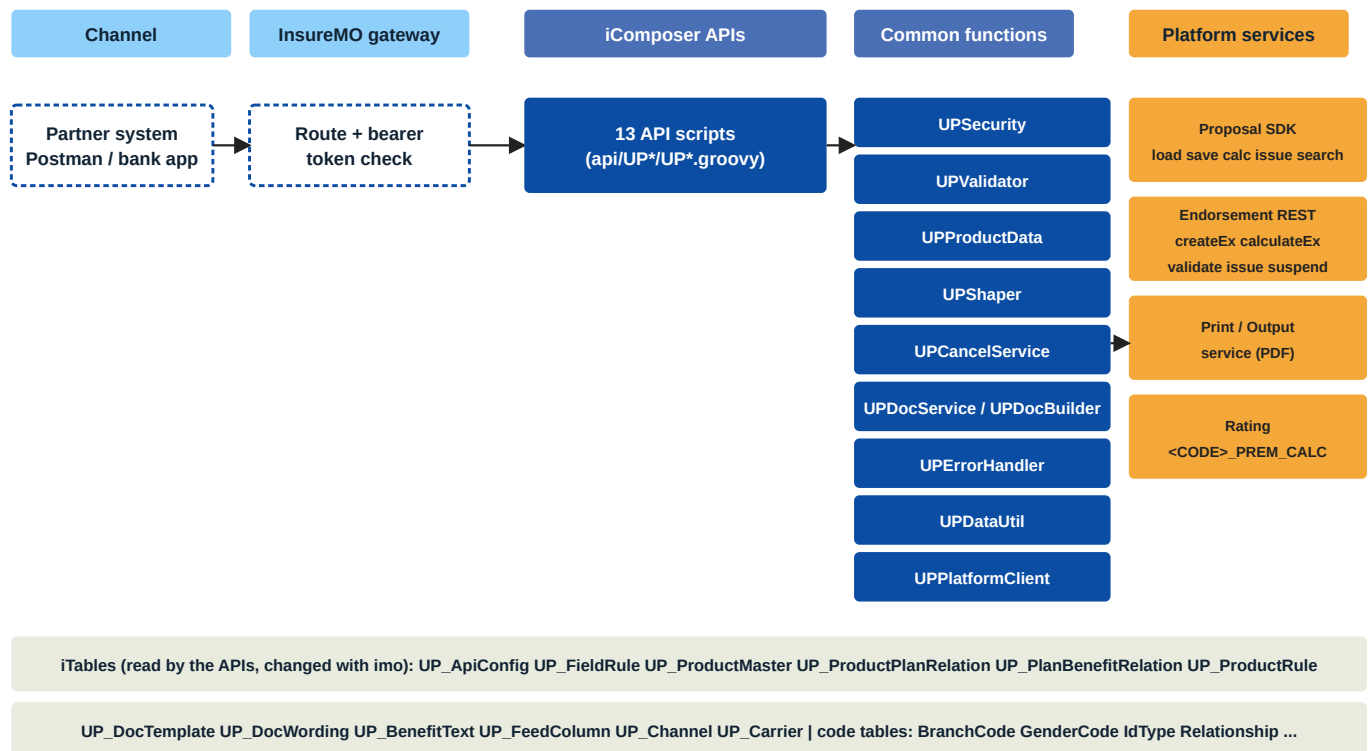
A set of 13 REST APIs (12 active, 1 retired) written as iComposer Groovy scripts on the InsureMO platform. They orchestrate the platform's proposal, policy, endorsement, search and print services for seven insurance products (UPPA001 Personal Accident, TL001 Term Life, FP001 Female Protection, HC001 Hospital Cash, CI001 Critical Illness, LP001 Life Protect, DH001 Domestic Helper; 11 variants, 57 plans). Partners such as Sohar Bank call the APIs; the APIs validate the request, call the platform and return a stable contract. Everything that a business user may want to change (validation rules, product data, refund rules, texts, formats, channels, insurers) is data in iTables, not code.

1.3 Glossary

Term	Meaning
MC	The current development tenant in which everything is built and tested (tenant code <code>uponly</code> , gateway <code>portal-gw.insuremo.com</code>). UAT is a separate tenant.
iComposer	InsureMO's low-code Groovy runtime. An API script is exposed as a REST route; a function (common service) is a shared Groovy class called with <code>getCommonService</code> .
iTables / data table	Tenant configuration tables read by the code (<code>UP_ApiConfig</code> , <code>UP_FieldRule</code> , ...).
Tech product / market product	The platform keeps a technical product (<code><CODE>_TECH</code> , holds the data dictionary tree) and a market product (<code><CODE></code>) per product code.
DD / binding	Data dictionary: the fields of the policy tree. A field must be bound to an object before the platform accepts or calculates it.
Checker / Approver	The two roles of the cancellation: the Checker creates the request, the Approver decides it.
ChannelCode	The partner that owns a record (SOHAR = Sohar Bank); sent on every request and saved on every proposal.
OrgCode	Issuing branch code; the platform validates it against the code table <code>PubBranch</code> (value 1).
Carrier / insurer	The insurer of a product: <code>CarrierCode</code> AFIC, name from table <code>UP_Carrier</code> .
Trace id	Platform request id, returned as <code>trace_id</code> and used to find the log of a call.

2. Architecture

2.1 Layers



Every API: `readInput` -> `validate` -> `platform call(s)` -> `shapeResponse`; any exception -> `UPErrHandler (status, code, message, trace_id)`

Figure 1: Architecture — channel, gateway, iComposer APIs, common functions, platform services and tables

Every API follows the same pipeline. The API script is thin: it reads the body through `UPSecurity.readInput`, validates it with `UPValidator.validate` (rules in table `UP_FieldRule` plus product and cross-field rules in code), calls the platform through the SDK clients (`ProposalSdkClient`) or the REST helper (`UPPlatformClient`), shapes the answer with `UPShaper.shapeResponse` and returns it. Any exception is turned into the standard error body by `UPErrHandler.handle`, which also sets the HTTP status. The tables are read-only from the code (the `TablesSdkClient` is read-only); they are changed with the `imo` tool by an administrator.

2.2 Design Rules

- Everything configurable lives in tables: a missing configuration key stops the request with `UP-CONFIG` (HTTP 500, generic message, detail in the log); there is no silent default. The only exceptions are the security baselines in `UPSecurity`, which have an optional table override.
- PA001 (the original product, config and code) is never read or changed by this solution: product data lives in separate `UP_` tables and the new products are new platform products. A snapshot tool (`snapshot.py`) proves this before and after every change.
- The caller's own bearer token is used for every platform call; there is no service account. The token is only sent to hosts listed in `UP_ApiConfig.AllowedHosts` (https only).
- Responses are shaped for the channel: platform technical fields may be present but are not part of the contract.
- One error contract on every API: `status`, `code`, `message`, `errors[]`, `api`, `trace_id`.

2.3 Request Lifecycle

- Gateway authenticates the bearer token (HTTP 401 on failure) and routes `/up/<path>` to the iComposer API.
- API script: `UPSecurity.readInput(RequestBody())` — body must be a JSON object, then every string is scanned for blocked characters (400 `INVALID_BODY` / `INVALID_CHARACTER`).

3. `UPValidator.validate(api, input)` — rules from `UP_FieldRule` for this API (required, type, length, pattern, range, enum, code table, conditional `WhenPath`), then product rules (age, term, premium mode, plan availability), `CarrierCode` match, beneficiary total. All problems are returned together as 400 `UP-VALIDATION`.
4. State and ownership checks for existing records: record exists (404), belongs to the channel (`requireChannel`), product matches, proposal still open (`requireOpenProposal`, 409).
5. Platform call(s): `ProposalSdkClient` (application, load, save, `persistCalculate`, issue, `queryPolicy`) or `UPPlatformClient` (endorsement REST) or the print service.
6. `UPShaper.shapeResponse`: commission switch, premium fields, payment info restored, `CarrierCode` / `CarrierName` added.
7. Return the map (HTTP 200) or, on any exception, `UPErrorHandler.handle(apiName, ex)`.

3. Source Layout and Tooling

3.1 Workspace Layout

```
ic/
  src/dev/Tenant/UP_PRODUCTS_API/
    api/<Name>/<Name>.groovy      13 API scripts (UPCancelPolicy is a retired stub, status inactive)
    function/<Name>/<Name>.groovy  10 common functions
    .metadata/icomposer/{api,function}/<Name>.metadata.json  per-profile server metadata (ids, version, md5,
path, method)
  uponly-build/                  build, test and report tooling (section 11)
    tables/                      the rows written to the UP_ tables (r_*.json), definitions (def_*.json),
local copies (rows_*.json)
    samples/                     recorded request / response of every flow step
    tests/ security/ carrier/     test cases and results
```

Module id 70, group id 1188491119; functions have scope `public` and status 1. APIs are POST except `UPLoadPolicy` (GET). All paths are `/up/<name>`: `get-plans`, `proposal`, `update-proposal`, `issue`, `load-policy`, `proposal-reject`, `doc_generation`, `application_doc_generation`, `cancel-check`, `cancel-approve`, `list-master-table`, `commission-query`, `generate-feedfile` (and the inactive `cancel-policy`).

3.2 The imo Tool

Task	Command (profile <code>portal:uponly</code> = MC)
Push one file (compiles on the server first)	<code>imo icomposer push current <file> --profile portal:uponly</code>
Preview a push	<code>... push current <file> --dry-run</code>
Create a function / API	<code>imo icomposer create function --name X --module-id 70 --group-id 1188491119 --status 1 --func-scope 1 --profile portal:uponly</code>
Pull the server copy	<code>imo icomposer reload ...</code> ; on a conflict choose <code>--prefer-local</code> or <code>--prefer-server</code>
Read a table / rows	<code>imo config datatable by-name --name UP_ApiConfig --json, ... record-list --id <tableId> --json</code>
Add rows	<code>imo config datatable record-batch-save --file rows.json --confirm</code> (insert only; the primary key <code>Id</code> must be unique)
Change one row	<code>imo config datatable record-save --file row.json --confirm</code> (include <code>RecordId</code>)
Create a table	<code>imo config datatable create --file def.json --confirm</code>
Find a call in the log	<code>imo log query</code> with the trace id from the response (section 9)

3.3 Groovy Rules of This Runtime

- The server type-checks Groovy statically on push. `@groovy.transform.Field` is not allowed and script-level variables are not visible inside methods: use a method that returns the constant (see `UPSecurity.blockedBaseline()`).
- `log` is not declared: use `org.slf4j.LoggerFactory.getLogger("Name")` (see `UPErrorHandler`).
- Shared per-request state goes into the request attributes (`RequestContextHolder`), as the table cache in `UPDataUtil.getRecords` does.
- A function that calls another one declares it with `(UPOther) getCommonService("UPOther")`; push the called function first when it is new.
- The platform REST client raises its own exception type; `UPPlatformClient` catches it and returns `{status, body}` so that callers decide.
- Keep each function focused; the largest file is `UPValidator` (333 lines).

3.4 Deployment Order of the Code

Dependencies run from the bottom up. Push in this order (the same order is used for UAT): `UPDataUtil`, `UPSecurity`, `UPErrorHandler`, `UPPlatformClient`, `UPValidator`, `UPProductData`, `UPShaper`, `UPDocBuilder`,

UPDocService, UPCancelService, then the 12 active APIs. Pushing a caller before a new callee fails the compile check.

4. Code Reference

4.1 Asset Inventory

Name	Kind	Route	Status	Lines	Server version	Calls (common functions)
UPApplicationDocGeneration	API	POST /up/application_doc_generation	active	24	7	UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator
UPCancelApprove	API	POST /up/cancel-approve	active	24	5	UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPCancelCheck	API	POST /up/cancel-check	active	25	5	UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPCancelPolicy	API	POST /up/cancel-policy	inactive	5	6	
UPCommissionQuery	API	POST /up/commission-query	active	136	10	UPDataUtil, UPDocBuilder, UPErrorHandler, UPProductData, UPSecurity, UPValidator
UPDocGeneration	API	POST /up/doc_generation	active	23	7	UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator
UPFeedFile	API	POST /up/generate-feedfile	active	178	9	UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator
UPGetPlans	API	POST /up/get-plans	active	104	7	UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator
UPIssue	API	POST /up/issue	active	51	9	UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator
UPListMasterTable	API	POST /up/list-master-table	active	37	7	UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPLoadPolicy	API	GET /up/load-policy	active	36	7	UPDataUtil, UPErrorHandler, UPShaper, UPValidator
UPProposal	API	POST /up/proposal	active	37	6	UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator
UPProposalReject	API	POST /up/proposal-reject	active	43	9	UPDataUtil, UPErrorHandler, UPSecurity, UPValidator
UPUpdateProposal	API	POST /up/update-proposal	active	58	9	UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator
UPCancelService	Function	-	active	303	7	UPDataUtil, UPErrorHandler, UPPlatformClient, UPProductData, UPValidator
UPDataUtil	Function	-	active	235	9	
UPDocBuilder	Function	-	active	195	3	UPDataUtil, UPProductData
UPDocService	Function	-	active	105	4	UPDataUtil, UPDocBuilder, UPErrorHandler, UPShaper, UPValidator
UPErrorHandler	Function	-	active	100	9	UPDataUtil
UPPlatformClient	Function	-	active	49	7	UPDataUtil

Name	Kind	Route	Status	Lines	Server version	Calls (common functions)
UPProductData	Function	-	active	202	4	UPDataUtil, UPSecurity, UPShaper
UPSecurity	Function	-	active	116	4	UPDataUtil
UPShaper	Function	-	active	126	3	UPDataUtil, UPProductData
UPValidator	Function	-	active	334	3	UPDataUtil, UPProductData

Total: 2546 lines of Groovy (781 in APIs, 1765 in functions).

4.2 Common Functions

UPCancelService (303 lines)

UPCancelService - cancellation in two steps (Checker, then Approver). * The request itself is the platform's own pending cancellation endorsement: the Checker creates it (the endorsement number is the RequestNo), the Approver approves it (calculate, validate, issue) or rejects it (suspend). No premium is calculated by the platform here; the refund is calculated by UPDataUtil.refundCalc from the product rules. Every code (statuses, types, paths) comes from UP_ApiConfig.

tables read: ProductMaster | SDK: ProposalSdkClient | config keys used: CancellInstallmentFlag, CancellInstallmentType, CancelPayMode, CancelReasonCode, CancelTypeFreeLook, CancelTypeProRata, DecisionApprove, EndoBasePath, EndoTypeCancel, PlatformErrorMaxLength, PolicyStatusInforce, ProposalLoadPath, RefundTypeFreeLook, RefundTypeProRata, RequestStatusApproved, RequestStatusPending, RequestStatusRejected | calls: UPDataUtil, UPErrorHandler, UPPlatformClient, UPProductData, UPValidator

Method	Parameters	Returns	Purpose
resolvePolicy	Map<String, Object> input	Map<String, Object>	The issued policy named by PolicyNo or ProposalNo, checked against the caller's channel. Returns null when it does not exist.
pendingCancellation	Map<String, Object> policy	Map<String, Object>	The pending cancellation endorsement of the policy, or null. Another kind of pending endorsement blocks the request.
cancellationDate	Map<String, Object> endorsement	LocalDate	Cancellation date used for the refund: the endorsement effective date, or the start of the policy for a free-look cancellation.
policyInfo	Map<String, Object> policy	Map<String, Object>	Policy information block shared by both responses.
policyStatus	Map<String, Object> policy	Map<String, Object>	
createRequest	Map<String, Object> policy, String effectiveDate, boolean freeLook, String reason	Map<String, Object>	Creates the pending cancellation endorsement (the request). No premium is calculated by the platform here.
suspend	Map<String, Object> endorsement	void	Withdraws a pending endorsement (reject, or before it is replaced on approval).
check	Map<String, Object> input	Map<String, Object>	Checker: creates (or returns) the pending cancellation request and shows premium and refund details.

Method	Parameters	Returns	Purpose
decide	Map<String, Object> input	Map<String, Object>	Approver: approves (cancels the policy through the platform) or rejects the pending request named by RequestNo.
refundCalc	Map<String, Object> policy, java.time.LocalDate cancelDate	Map<String, Object>	Refund rule for a cancellation: full premium within the free-look period (counted from the commencement date), otherwise the percentage configured in UP_ProductRule (ProRataRefundPercent) of the pro-rata premium. Premium = plan premium on the risk (before tax). FreeLookDays and ProRataRefundPercent come from ProductRule.

UPDataUtil (235 lines)

UPDataUtil - core helpers: configuration (UP_ApiConfig), table access, flag values, formats, error helpers.

tables read: UP_ApiConfig | SDK: TablesSdkClient | config keys used: AllowedHosts, DateFormatDoc, FalseValue, MoneyFormat, MoneyScale, NumberFormat, PercentBase, RoundingMode, TrueValue

Method	Parameters	Returns	Purpose
fail	String message	void	
str	Object value	String	
yes		String	Flag values come from UP_ApiConfig (TrueValue / FalseValue), never from literals in code.
no		String	
isYes	Object value	boolean	
isNo	Object value	boolean	
requireAllowedHost	String url	void	The URL's host must be listed in UP_ApiConfig AllowedHosts and the scheme must be https before a bearer token is sent to it.
fill	String text, Map<String, String> values	String	Replaces {name} markers in a configured text.
roundingMode		java.math.RoundingMode	
moneyScale		int	
percentBase		BigDecimal	
configMap	String scope	Map<String, String>	Configuration value from UP_ApiConfig (scope = product code or ""). A missing key is a configuration error, never a silent default.
cfgValue	Map<String, String> config, String key	String	
cfg	String key, String scope	String	
pathValue	Object root, String path	Object	Value at a path such as PolicyLobList[0].PolicyRiskList[0].DateOfBirth.
fieldError	String field, String code, String message	Map<String, Object>	
throwValidation	List<Map<String, Object>> errors	void	
parseDate	Object value	LocalDate	

Method	Parameters	Returns	Purpose
getRecords	String tableName	List<Map<String, Object>>	All records of a datatable as plain maps. Read once per request (kept in the request attributes), so data changes are visible on the next request.
readRecords	String tableName	List<Map<String, Object>>	
filterRecords	String tableName, Map<String, Object> filters	List<Map<String, Object>>	
toDecimal	Object value	BigDecimal	
fmtDate	Object value	String	Display date, pattern from UP_ApiConfig DateFormatDoc.
fmtMoney	Object value	String	Display amount, pattern from UP_ApiConfig MoneyFormat.
fmtNumber	String v	String	Display number, pattern from UP_ApiConfig NumberFormat.

UPDocBuilder (195 lines)

UPDocBuilder - print data for the application form / certificate, document wording, commission view.

tables read: UP_BenefitText, UP_DocWording | config keys used: PlanLabelPrefix | calls: UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
describe	String tableName, Object id	String	
benefitText	Map<String, Object> benefit	String	Benefit limit wording from UP_DocWording (ModeCode = BASIS: <LimitBasis>, falling back to the BASIS:Default row of the same table).
docBenefit	String productCode, String code	List<String>	Official English / Arabic benefit names (UP_BenefitText): [english, arabic]. A row is required for every benefit that is printed.
premiumWording	String modeCode, String term, String premium	Map<String, Object>	English / Arabic wording of the premium frequency, premium text and period of cover for a premium mode (UP_DocWording).
buildPrintData	Map<String, Object> policy, String docType	Map<String, Object>	
rate	Object commission, Object premium	String	
commissionOfPolicy	Map<String, Object> policy	Map<String, Object>	Commission view of one loaded policy (commission is read from the stored policy, not from configuration).

UPDocService (105 lines)

UPDocService - builds the application form or the policy certificate as a PDF through the print service. Shared by UPDocGeneration and UPApplicationDocGeneration. Every setting (template version, print URL, export type, storage, mode, file name) comes from UP_ApiConfig, the template name from UP_DocTemplate.

tables read: DocTemplate | SDK: ProposalSdkClient | config keys used: DocContentType, DocFileNamePattern, DocTypeApplication, DocTypePolicy, PrintExportType, PrintMode, PrintStorage, PrintUrl, TemplateVersion | calls: UPDataUtil, UPDocBuilder, UPErrorHandler, UPShaper, UPValidator

Method	Parameters	Returns	Purpose
generate	String apiName, String docType, Map<String, Object> input	Object	Returns the PDF (HttpEntity), the print data when PreviewData is the TrueValue, or a 404 body.

UPErrorHandler (100 lines)

UPErrorHandler - turns an exception into a structured error response and sets the HTTP status. Validation problems return 400, state conflicts 409, platform API errors keep their status; configuration problems and anything else are 500 with a generic table-driven message (the detail goes to the log, the caller gets the trace id).

config keys used: *MsgConfigError, MsgInvalidNumber, MsgPlatformInput, MsgServerError* | calls: *UPDataUtil*

Method	Parameters	Returns	Purpose
configText	String key, String technical	String	Message text from UP_ApiConfig; the technical fallback is used only when the configuration itself cannot be read.
isInternalText	String message	boolean	True when a platform message carries database or framework internals (index names, class names, parser positions) that must not reach the caller.
handle	String apiName, Exception ex	Map<String, Object>	
notFound	String apiName, String message	Map<String, Object>	

UPPlatformClient (49 lines)

UPPlatformClient - calls platform services (endorsement) with the caller's own token. Returns { status: <http status>, body: <response text> } and never throws for HTTP or platform errors.

config keys used: *GatewayUrl* | calls: *UPDataUtil*

Method	Parameters	Returns	Purpose
call	String method, String path, Object body	Map<String, Object>	
parse	Map<String, Object> response	Map<String, Object>	

UPProductData (202 lines)

UPProductData - product master data (variants, rules, plans, benefits) and preparation of the proposal payload.

tables read: *ProductMaster, ProductPlanRelation, ProductRule, UP_Carrier, UP_PlanBenefitRelation* | config keys used: *RiskElementCode* | calls: *UPDataUtil, UPSecurity, UPShaper*

Method	Parameters	Returns	Purpose
getVariant	String productCode, String subcode	Map<String, Object>	
carrierOf	String productCode, String subcode	Map<String, Object>	Insurer (carrier) of a product variant: the code is on the product (UP_ProductMaster.CarrierCode), the name comes from UP_Carrier. A product without a carrier, or a carrier that is missing or inactive in UP_Carrier, is a configuration error (UP-CONFIG), never a default.
addCarrier	Map<String, Object> target, String productCode, String subcode	void	Adds CarrierCode and CarrierName (the insurer of the product) to a response map.
getRule	String productCode, String subcode	Map<String, Object>	
allowedModes	Map<String, Object> rule	List<String>	
getPlanRows	String productCode, String subcode, String premiumModeCode	List<Map<String, Object>>	Plan rows (premium + commission) for the variant and premium mode.
getBenefitRows	String productCode, String subcode, String planCode	List<Map<String, Object>>	

Method	Parameters	Returns	Purpose
buildPlanView	String productCode, String subcode, String premiumModeCode	List<Map<String, Object>>	Plans for the front end: premium and benefits only, never commission.
prepareProposal	Map<String, Object> input	Map<String, Object>	Validates the request against product data and fills the authoritative values (plan premium, commission, coverages when none are supplied). Returns the payload ready for the native proposal SDK.

UPSecurity (116 lines)

UPSecurity - request hygiene shared by every API. readInput the body must be a JSON object (anything else is a 400, never a 500); it is scanned before any API logic runs scan blocked characters in every text value, extra allow-list for free-text fields (400 INVALID_CHARACTER) stripServerOwned fields the server owns (policy number, ids, status) are never taken from the caller defaultOrgCode a proposal always carries a branch that the platform code table PubBranch accepts The baselines below are security controls: when UP_ApiConfig has the key (BlockedCharacters, FreeTextFields, FreeTextPattern, ServerOwnedFields, OrgCode) the table value is used, otherwise the built-in baseline applies (never an error).

calls: UPDataUtil

Method	Parameters	Returns	Purpose
blockedBaseline		String	Control characters and the characters used for script, template, shell and expression injection.
freeTextBaseline		String	Free-text fields: letters of any script, digits, space and . , ' - / @ + () only.
freeTextFieldsBaseline		String	
serverOwnedBaseline		String	
orgCodeBaseline		String	
maxFindings		int	
setting	String key, String baseline	String	Optional table override of a baseline.
readInput	Object body	Map<String, Object>	The request body as a map, already checked by scan(). Null is an empty request; an array, text or number is a 400.
scan	Map<String, Object> input	void	Rejects blocked characters in any text value, and anything outside the allow-list in the free-text fields. The value is never echoed.
scanValue	Object value, String path, String key, Pattern blocked, Pattern freeText, Set<String> freeFields, List<Map<String, Object>> errors, UPDataUtil util	void	
stripServerOwned	Map<String, Object> input	void	Removes the top-level fields the server owns. The platform sets the proposal status itself; a status sent by the caller made the platform fail (HTTP 500 / 502), so it is dropped.
defaultOrgCode	Map<String, Object> input	void	A branch the platform accepts (code table PubBranch); the caller's own OrgCode is kept.

UPShaper (126 lines)

UPShaper - shapes the responses returned to callers (commission, premium fields, payment info).

config keys used: InstalmentAmount, InstalmentFeeSeq, InstalmentPayRate, InstalmentPeriodCount, InstalmentPeriodSeq, PolicyStatusEffective, ReturnCommission, SequenceNumberStart, ZeroPremiumFields,

Method	Parameters	Returns	Purpose
stripCommission	Object node	Object	Removes commission from a policy / proposal body before it is returned to a front-end caller.
restorePayment	Object body	Object	Rebuilds PolicyPaymentInfoList from the flattened payment fields.
returnCommission		boolean	Commission is returned only while UP_ApiConfig ReturnCommission equals the TrueValue.
enrichPremiumFields	Map<String, Object> map	void	Adds the premium structure fields the PA001 responses carry (zero where the level is not rated) and the plan row premium / commission.
shapeResponse	Object body	Object	Response shaping for every front-end facing API: same structure as the PA001 flow, payment info in its original shape.

UPValidator (334 lines)

UPValidator - request validation: field rules (UP_FieldRule), cross-field and product rules, state / channel checks.

tables read: ProductMaster, UP_FieldRule | config keys used: BeneficiaryShareTotal, EffectiveDateMinOffsetDays, MasterTables, NonSingleTermYears, PatternMaxInput, PatternMaxLength, ProposalStatusIssued, ProposalStatusRejected, SinglePremiumModeCode | calls: UPDataUtil, UPProductData

Method	Parameters	Returns	Purpose
requireOpenProposal	Map<String, Object> stored, String action	void	A proposal can only be updated / issued / rejected while it is open (status codes come from UP_ApiConfig).
requireProductMatch	Map<String, Object> stored, String productCode, String subcode	void	The stored proposal / policy must belong to the product the caller names.
requireChannel	Map<String, Object> stored, String channelCode	void	The stored proposal / policy must have been created by the channel that calls (ChannelCode is saved on every proposal).
codeExists	String table, String column, String value	boolean	
validate	String api, Map<String, Object> input	void	Field level validation. Rules come from UP_FieldRule (per API) and product rules from UP_ProductRule. All problems are collected and returned together as HTTP 400 with errors[].
crossFieldErrors	String api, Map<String, Object> input, boolean productFieldBad	List<Map<String, Object>>	Rules that need several fields or product data. Only runs on values that already passed the field rules.
entryErrors	Map<String, Object> rule, String dateOfBirth, String effectiveDate, String premiumModeCode, String policyTerm	List<Map<String, Object>>	Age, term and premium mode checks from UP_ProductRule, returned as field errors.

4.3 APIs

UPApplicationDocGeneration — POST /up/application_doc_generation (24 lines)

Generate the application form of a proposal as a PDF (same request as the PA001 flow). Request: { "proposalNo": "...", "ProductSubcode": "1" } The template is looked up in UP_DocTemplate by ProductCode + ProductSubcode + DocType.

calls: UPDataUtil, UPDocService, UPErrHandler, UPSecurity, UPValidator | config keys: DocTypeApplication

UPCancelApprove — POST /up/cancel-approve (24 lines)

Cancellation - Approver. Approves (cancels the policy) or rejects the pending request created by the Checker. Request: { "ChannelCode": "SOHAR", "PolicyNo" | "ProposalNo": "...", "RequestNo": "<from the Checker>", "Decision": "APPROVE" | "REJECT" } Response: RequestNo, RequestStatus, PolicyInfo, PolicyStatus, RefundPremium.

calls: UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator

UPCancelCheck — POST /up/cancel-check (25 lines)

Cancellation - Checker. Creates the pending cancellation request for an issued policy and shows what an approval would do. Request: { "ChannelCode": "SOHAR", "PolicyNo" | "ProposalNo": "...", "CancellationDate": "2026-10-20", "CancelReason": "..." } Response: RequestNo, RequestStatus, PolicyInfo, PolicyStatus (current), PremiumDetails, RefundDetails. Repeating the call for a policy that already has a pending request returns that request (RequestExists = TrueValue).

calls: UPCancelService, UPDataUtil, UPErrorHandler, UPSecurity, UPValidator

UPCancelPolicy — POST /up/cancel-policy (5 lines)

Retired (inactive). Cancellation is now done in two steps: UPCancelCheck (/up/cancel-check) and UPCancelApprove (/up/cancel-approve).

UPCommissionQuery — POST /up/commission-query (136 lines)

Commission query - the only API that returns commission (all other APIs hide it). * 1. By policy: { "PolicyNo": "POFP00100000013" } 2. By configuration: { "ProductCode": "FP001", "ProductSubcode": "1", "PlanId": "2" (optional) } 3. By period: { "ProductCode": "FP001" (optional), "FromDate": "2026-10-01", "ToDate": "2026-10-31", "PageNo": 1, "PageSize": 50 } * Commission is configured per product / variant / plan in the ProductPlanRelation table (column Commission). It is written onto the policy when the premium is calculated, so a policy keeps the commission it was issued with.

calls: UPDataUtil, UPDocBuilder, UPErrorHandler, UPProductData, UPSecurity, UPValidator | SDK: ProposalSdkClient | tables: ProductPlanRelation | config keys: CommissionDefaultPageNo, CommissionDefaultPageSize, DayEndTime, DayStartTime, ProposalStatusIssued, SearchModule, SearchSortField, SearchSortType

UPDocGeneration — POST /up/doc_generation (23 lines)

Generate the application form or the policy certificate as a PDF. Request: { "DocType": "Application" | "Policy", "ProposalNo": "...", "PolicyNo": "..." }

calls: UPDataUtil, UPDocService, UPErrorHandler, UPSecurity, UPValidator

UPFeedFile — POST /up/generate-feedfile (178 lines)

Generate the policy feed file (Excel by default, JSON when OutputFormat = "JSON"). Request: { "Conditions": {"ProductCode": "FP001"}, "FromRangeConditions": {"EffectiveDate": "..."}, "ToRangeConditions": {"EffectiveDate": "..."}, "SearchType": "Policy", "PageNo": 1, "PageSize": 50, "OutputFormat": "XLSX" | "JSON" } Commission is not part of the feed file.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient | tables: UP_FeedColumn | config keys: FeedContentType, FeedFileName, FeedMaxPages, FeedMaxRecords, FeedSheetName, ProposalStatusEntry, ProposalStatusIssued, SearchModule, SearchSortField, SearchSortType

UPGetPlans — POST /up/get-plans (104 lines)

Get plans for a product variant. Same shape as the PA001 flow: the request is the policy body (ProductCode + ProductSubcode, insured details), the response is that policy with every plan of the variant in PolicyRiskList[].PlanList (PlanId, PlanName, DuePremium, Commission, PolicyCoverageList). Optional: PremiumModeCode, PolicyTerm. Age and term rules are checked when the date of birth is supplied.

calls: UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator | config keys: BusinessCateCode, CustomerFlagNo, IsAutoAttachedForm, IsPremiumCalcSuccess, PolicyStatusEffective, RiskElementCode, SequenceNumberStart, ZeroPremiumFields, ZeroPremiumValue

UPIssue — POST /up/issue (51 lines)

Issue a policy from a proposal. Request: { "ProposalNo": "...", "ProductCode": "..." }.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient

UPListMasterTable — POST /up/list-master-table (37 lines)

List master data for the front-end dropdowns. Request: { "DataTableName": "Relationship", "Id": "2" (optional) }.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | config keys: MasterTables

UPLoadPolicy — GET /up/load-policy (36 lines)

Load a policy (policyNo) or a proposal (proposalNo). Commission is removed from the response.

calls: UPDataUtil, UPErrorHandler, UPShaper, UPValidator | SDK: ProposalSdkClient

UPProposal — POST /up/proposal (37 lines)

Create a proposal (application). The request uses the native policy JSON (PolicyLobList / PolicyRiskList / PlanList / PolicyCoverageList / BeneficiaryList ...). The plan premium is taken from product data; coverages are filled from the plan when not supplied.

calls: UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient

UPProposalReject — POST /up/proposal-reject (43 lines)

Reject a proposal (checker action). Request: { "ProposalNo", "ProposalRejectDesc" }.

calls: UPDataUtil, UPErrorHandler, UPSecurity, UPValidator | SDK: ProposalSdkClient | config keys: SuccessStatusText

UPUpdateProposal — POST /up/update-proposal (58 lines)

Update an existing proposal. ProposalNo is required; the policy identifiers are loaded from the stored proposal.

calls: UPDataUtil, UPErrorHandler, UPProductData, UPSecurity, UPShaper, UPValidator | SDK: ProposalSdkClient

5. API Walkthroughs

What each API does in code, in the order it does it. Names in backticks are functions or methods listed in section 4.

API	Flow in code
UPGetPlans	Validates the body (<code>validator.validate("UPGetPlans")</code>): product, variant, premium mode, term, age from <code>DateOfBirth</code>). Reads the variant (<code>UPProductData.getVariant</code>) and rule (<code>getRule</code>) and builds the <code>PlanList</code> directly from the tables: <code>getPlanRows</code> (<code>UP_ProductPlanRelation</code> : plan id, name, premium, commission) and <code>getBenefitRows</code> (<code>UP_PlanBenefitRelation</code> : code, description, sum insured) — no platform call. The premium shown here is therefore the table premium; the proposal premium is calculated by the platform rating <code><CODE>_PREM_CALC</code> from the same tables, so the two stay equal as long as the data is consistent. Defaults of the policy skeleton (status, zero premium fields, flags, customer flags) come from <code>UP_ApiConfig</code> (<code>PolicyStatusEffective</code> , <code>ZeroPremiumFields</code> , <code>CustomerFlagNo</code> , ...). It does not call the platform and does not use <code>shapeResponse</code> , so <code>CarrierCode</code> / <code>CarrierName</code> are added with <code>products.addCarrier</code> before the return.
UPProposal	Rejects <code>ProposalNo</code> (409-style 400), validates, then <code>UPProductData.prepareProposal</code> : drops client <code>PlanList</code> and <code>DuePremium</code> (the platform rating builds plan, covers, premium and commission), removes server-owned fields and defaults <code>OrgCode</code> (<code>UPSecurity</code>), flattens <code>PolicyPaymentInfoList</code> into <code>Payment*</code> policy fields (the platform instalment rating would otherwise re-round the premium) and returns the payload. The API then calls <code>proposalApi().newApplicationRequestBuilder()</code> and returns <code>shaper.shapeResponse</code> , which rebuilds the payment structure, applies the commission switch and adds the insurer.
UPUpdateProposal	Same preparation as create. Loads the stored proposal by <code>ProposalNo</code> (404), checks channel, product match and that it is open (409), copies <code>PolicyId</code> , <code>PolicyElementId</code> and <code>VersionSeq</code> of the stored record into the payload, calls <code>newSaveRequestBuilder()</code> and then <code>newPersistCalculateRequestBuilder()</code> because the save drops the calculated plan row.
UPIssue	Loads the proposal (404), checks channel, product match (<code>MISMATCH</code>) and open state (409). When the optional request field <code>Recalculate = Y</code> it first runs <code>persistCalculate</code> on the stored proposal (not part of the public specification). Then <code>newIssuePolicyRequestBuilder</code> issues it and the shaped policy with the new <code>PolicyNo</code> is returned.
UPLoadPolicy	GET. Exactly one of <code>policyNo</code> / <code>proposalNo</code> (400); loads with code descriptions (<code>withCodeDesc</code>), 404 when missing, <code>requireChannel</code> , <code>shapeResponse</code> .
UPProposalReject	Loads the proposal (404), checks channel and open state (409), calls <code>newRejectRequestBuilder</code> with a <code>RejectVo</code> (proposal number and reason) and returns <code>{status: SUCCESS (SuccessStatusText), message: <the reason>}</code> .
UPDocGeneration / UPApplicationDocGeneration	Thin wrappers over <code>UPDocService.generate</code> : resolve the template name from <code>UP_DocTemplate</code> (product, sub-product, <code>DocType</code>), build the print data with <code>UPDocBuilder</code> (wording from <code>UP_DocWording</code> , benefit names from <code>UP_BenefitText</code> , formats from <code>UP_ApiConfig</code>), call the print service at <code>PrintUrl</code> (host checked against <code>AllowedHosts</code>) with the caller's token and return the PDF (or the print data when <code>PreviewData = Y</code>).
UPCancelCheck / UPCancelApprove	See section 8 for the complete sequence: <code>UPCancelService.check</code> and <code>decide</code> .
UPListMasterTable	Allows only tables listed in <code>UP_ApiConfig.MasterTables</code> ; returns all records or the one with <code>Id</code> (commission column removed).
UPCommissionQuery	Three modes: by <code>PolicyNo</code> (<code>UPDocBuilder.commissionOfPolicy</code>), by product configuration (rows of <code>UP_ProductPlanRelation</code> with premium, commission and rate), or for a period (search on the platform, load each policy, keep only the caller's channel, page totals). Page size is capped by <code>CommissionMaxPageSize</code> .
UPFeedFile	Searches policies with the platform query API (period, product, paging), loads each hit, keeps the caller's channel, shapes it and writes the columns defined in <code>UP_FeedColumn</code> (heading + source path, alternatives separated by <code> </code>) to an Excel sheet or to JSON. Capped by <code>FeedMaxRecords</code> / <code>FeedMaxPages</code> (400 <code>RANGE_T00_LARGE</code>).

6. Data and Configuration Reference

6.1 Tables

Table	Table id (MC)	Rows	Columns	Purpose	Read by
UP_ApiConfig	1188575596	81	Api, Scope, Key, Value	Key / value configuration of the code (Api, Scope, Key, Value). configMap(scope) merges * rows with product-specific rows.	UPDataUtil.cfg / cfgValue, every function
UP_FieldRule	1188575333	138	Api, Path, Label, Required, DataType, MinLength, MaxLength, Pattern, MinValue, MaxValue, CodeTable, CodeColumn, EnumValues, AppliesTo, WhenPath, WhenValue	Validation rules per API and field path: Required, DataType, MinLength, MaxLength, Pattern, MinValue, MaxValue, EnumValues, CodeTable + CodeColumn, WhenPath + WhenValue (conditional), AppliesTo.	UPValidator.validate
UP_ProductMaster	1188475672	11	Description, LOB, IMOProductCode, IsRenewable, IsActive, IsFlatRate, Premium, SumInsured, HasPlans, CarrierCode, CarrierProductCode, CarrierProductName, IsResident, BenefitRelationTableName, LobId, CarrierName	Product variants (IMOProductCode, CarrierProductCode = sub-product, name, LOB, CarrierCode, IsActive).	UPProductData.getVariant, carrierOf
UP_ProductPlanRelation	1188475676	57	IMOProductCode, IMOPlanCode, IMOPlanName, CarrierCode, CarrierProductCode, CarrierProductName, CarrierPlanCode, CarrierPlanName, IsActive, Premium, SumInsured, Commission, PremiumModeCode	Plans per variant with premium, commission, premium mode.	UPProductData, UPGetPlans, commission configuration view
UP_PlanBenefitRelation	1188474890	148	IMOProductCode, IMOPlanCode, CarrierCode, CarrierProductCode, CarrierPlanCode, BenefitCode, BenefitDesc, BenefitLevel, ParentBenefitCode, BenefitLimit, BenefitLimitDesc, IsMainBenefit, IsAdditionalBenefit, Notes, IsApplicable, LimitBasis, MaxDays	Covers and sums insured per plan.	UPProductData, UPGetPlans, documents

Table	Table id (MC)	Rows	Columns	Purpose	Read by
UP_Product Rule	1188475684	11	ProductCode, ProductSubcode, PremiumModes, MinTerm, MaxTerm, MinAge, MaxAge, MaxEntryAge, MaxExitAge, MandatoryDocuments, SinglePremiumRule, FreeLookDays, ProRataRefundPercent	Per variant: MinAge, MaxAge, MaxEntryAge, MaxExitAge, MinTerm, MaxTerm, PremiumModes, FreeLookDays, ProRataRefundPercent, mandatory documents.	UPValidator.entryErrors, UPCancelService.refundCalc
UP_DocTemplate	1188475689	22	DocumentCode, TemplateName, FileName, ProductCode, ProductSubcode, DocType	Output template name per product, sub-product and document type.	UPDocService
UP_DocWording	1188556760	27	ModeCode, Key, Value	Wording for premium modes and benefit limit bases (BASIS: *), English / Arabic.	UPDocBuilder
UP_BenefitText	1188575620	21	ProductCode, BenefitCode, NameEn, NameAr	Official English / Arabic benefit names per product.	UPDocBuilder
UP_FeedColumn	1188591810	29	Sequence, Heading, Source	Columns of the policy feed file (Sequence, Heading, Source).	UPFeedFile
UP_Channel	1188591925	1	ChannelCode, Description, IsActive	Registered channels (ChannelCode, Description, IsActive).	UPValidator (ChannelCode rule)
UP_Carrier	1188663015	1	CarrierCode, CarrierName, IsActive	Insurers (CarrierCode, CarrierName, IsActive); linked from UP_ProductMaster.CarrierCode.	UPProductData.carrierOf

The code maps a few logical names to the `UP_` tables (`UPDataUtil.readRecords`): `ProductMaster`, `ProductPlanRelation`, `UP_PlanBenefitRelation`, `ProductRule` and `DocTemplate`. Tables are read once per request and cached in the request attributes, so a changed row is visible on the next request without any deployment.

6.2 Configuration Keys (UP_ApiConfig)

Every key with its current MC value and the functions that read it. Environment specific keys are marked in section 13.

Key	Value (MC)	Used in
AllowedHosts	portal-gw.insuremo.com	UPDataUtil
BeneficiaryShareTotal	100	UPValidator
BusinessCateCode	1	UPGetPlans
CancelInstallmentFlag	Y	UPCancelService
CancelInstallmentType	1	UPCancelService
CancelPayMode	30	UPCancelService
CancelReasonCode	1	UPCancelService
CancelTypeFreeLook	31	UPCancelService
CancelTypeProRata	32	UPCancelService
CommissionDefaultPageNo	1	UPCommissionQuery
CommissionDefaultPageSize	50	UPCommissionQuery
CommissionMaxPageSize	50	(read through a pattern or documented only)
CustomerFlagNo	N	UPGetPlans

Key	Value (MC)	Used in
DateFormatDoc	dd/MM/yyyy	UPDataUtil
DayEndTime	T23:59:59	UPCommissionQuery
DayStartTime	T00:00:00	UPCommissionQuery
DecisionApprove	APPROVE	UPCancelService
DecisionReject	REJECT	(read through a pattern or documented only)
DocContentType	application/pdf	UPDocService
DocFileNamePattern	{docType}_{number}.pdf	UPDocService
DocTypeApplication	Application	UPApplicationDocGeneration, UPDocService
DocTypePolicy	Policy	UPDocService
EffectiveDateMinOffsetDays	0	UPValidator
EndoBasePath	/platform/endo/core/endo/v1	UPCancelService
EndoTypeCancel	3	UPCancelService
FalseValue	N	UPDataUtil
FeedContentType	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	UPFeedFile
FeedFileMaxPageSize	50	(read through a pattern or documented only)
FeedFileName	Policy_Feed.xlsx	UPFeedFile
FeedMaxPages	40	UPFeedFile
FeedMaxRecords	2000	UPFeedFile
FeedSheetName	PolicyFeed	UPFeedFile
GatewayUrl	https://portal-gw.insuremo.com	UPPlatformClient
InstalmentAmount	0	UPShaper
InstalmentFeeSeq	1	UPShaper
InstalmentPayRate	1	UPShaper
InstalmentPeriodCount	1	UPShaper
InstalmentPeriodSeq	1	UPShaper
IsAutoAttachedForm	Y	UPGetPlans
IsPremiumCalcSuccess	Y	UPGetPlans
MasterTables	BranchCode, CustomerType, GenderCode, Id Type, InsMaritalStatus, Plan, Pre...	UPListMasterTable, UPValidator
MoneyFormat	#,##0.00	UPDataUtil
MoneyScale	2	UPDataUtil
MsgConfigError	The service is not configured correctly. Please contact support and...	UPErrorHandler
MsgInvalidNumber	A numeric value in the request is not valid.	UPErrorHandler
MsgServerError	The request could not be processed. Please contact support and quot...	UPErrorHandler
NonSingleTermYears	1	UPValidator
NumberFormat	#,##0.##	UPDataUtil
PatternMaxInput	500	UPValidator
PatternMaxLength	200	UPValidator
PercentBase	100	UPDataUtil
PlanLabelPrefix	PLAN-	UPDocBuilder
PlatformErrorMaxLength	300	UPCancelService

Key	Value (MC)	Used in
PolicyStatusEffective	2	UPGetPlans, UPShaper
PolicyStatusInforce	2	UPCancelService
PrintExportType	pdf	UPDocService
PrintMode	sync	UPDocService
PrintStorage	not_store	UPDocService
PrintUrl	https://portal-gw.insuremo.com/mo-fo/1.0/print/v3/printtask/generate	UPDocService
ProposalLoadPath	/platform/proposal/v1/load	UPCancelService
ProposalStatusEntry	1	UPFeedFile
ProposalStatusIssued	3	UPCommissionQuery, UPFeedFile, UPValidator
ProposalStatusRejected	4	UPValidator
RefundTypeFreeLook	FREE_LOOK	UPCancelService
RefundTypeProRata	PRO_RATA	UPCancelService
RequestStatusApproved	APPROVED	UPCancelService
RequestStatusPending	PENDING	UPCancelService
RequestStatusRejected	REJECTED	UPCancelService
ReturnCommission	Y	UPShaper
RiskElementCode	R00001	UPGetPlans, UPProductData
RoundingMode	HALF_UP	UPDataUtil
SearchModule	Policy	UPCommissionQuery, UPFeedFile
SearchSortField	index_time	UPCommissionQuery, UPFeedFile
SearchSortType	desc	UPCommissionQuery, UPFeedFile
SequenceNumberStart	1	UPGetPlans, UPShaper
SinglePremiumModeCode	1	UPValidator
SuccessStatusText	SUCCESS	UPProposalReject
TemplateVersion	1	UPDocService
TrueValue	Y	UPDataUtil
ZeroPremiumFields	AdjustedPremium,AnnualPremium,BeforeVa tPremium,GrossPremium,Standar...	UPGetPlans, UPShaper
ZeroPremiumValue	0	UPGetPlans, UPShaper

6.3 Field Rules (UP_FieldRule)

API	Rules
UPApplicationDocGeneration	3
UPCancelApprove	5
UPCancelCheck	5
UPCommissionQuery	9
UPDocGeneration	6
UPFeedFile	8
UPGetPlans	8
UPIssue	3
UPListMasterTable	3
UPLoadPolicy	3
UPProposal	40

API	Rules
UPProposalReject	4
UPUpdateProposal	41

A rule row is evaluated by `UPValidator.validate`: Path uses indexes (`PolicyLobList[0].PolicyRiskList[0].InsuredName`); Required Y/N; `DataType` string / integer / decimal / date / enum; `Pattern` is a regular expression (length and input size are capped by `PatternMaxLength` / `PatternMaxInput`); `CodeTable` + `CodeColumn` check the value against a table (`UP_Channel`, `UP_Carrier`, `BranchCode`, ...); `WhenPath` + `WhenValue` make the rule conditional (for example `PolicyNo` is required when `DocType = Policy`); `AppliesTo` limits a rule to a product. Changing a rule is a row change and needs no deployment.

6.4 Security Baselines (UPSecurity)

Control	Baseline in code	Optional override key in UP_ApiConfig
Blocked characters in any text value	control characters and <code>< > \$ { } \ ^ ~ ; `</code>	BlockedCharacters
Free-text fields	InsuredName, Address, BeneficiaryName, CancelReason, OtherCancelReason, ProposalRejectDesc	FreeTextFields
Free-text allow-list	letters of any script, digits, space and <code>. , ' - / @ + ()</code>	FreeTextPattern
Server-owned fields removed from a proposal	PolicyNo, PolicyId, PolicyElementId, ProposalStatus, PolicyStatus	ServerOwnedFields
Default OrgCode	1	OrgCode

These are the only values that have a built-in default; a missing override key never fails a request. Moving them fully into tables (and treating a missing key as UP-CONFIG) is on the backlog.

7. Platform Configuration (Products)

7.1 What Exists on the Platform

For each of the seven product codes the platform holds a tech product `<CODE>_TECH` and a market product `<CODE>` (ids in the deployment plan), the policy data dictionary tree (Policy, PolicyLob, PolicyRisk R00001, Plan, PolicyCoverage, Beneficiary), a premium rating definition `<CODE>_PREM_CALC` bound to the market product, the plan and cover data in the `UP_` tables, and the output templates named in `UP_DocTemplate` (22 rows).

7.2 Data Dictionary Bindings

The new products bind only the fields the flows need (24 policy-level, 17 tech-level, 8 risk-level, 4 plan, 6 coverage, 4 beneficiary fields) at product level (RecordUsage 5). PA001 binds far more (408 policy-level fields). The cancellation endorsement rating additionally reads delta fields; `fix_endo_fields.py` bound the ones the platform reported as missing (`StandardGrossPremiumDelta`, `AnnualPremiumDelta`, `ShortRate` on the risk and coverage objects) on the new tech products only. The script never creates `VAT` (there is no VAT in this solution) and refuses to touch any context that is not one of the seven tech products.

7.3 How the Products Were Built

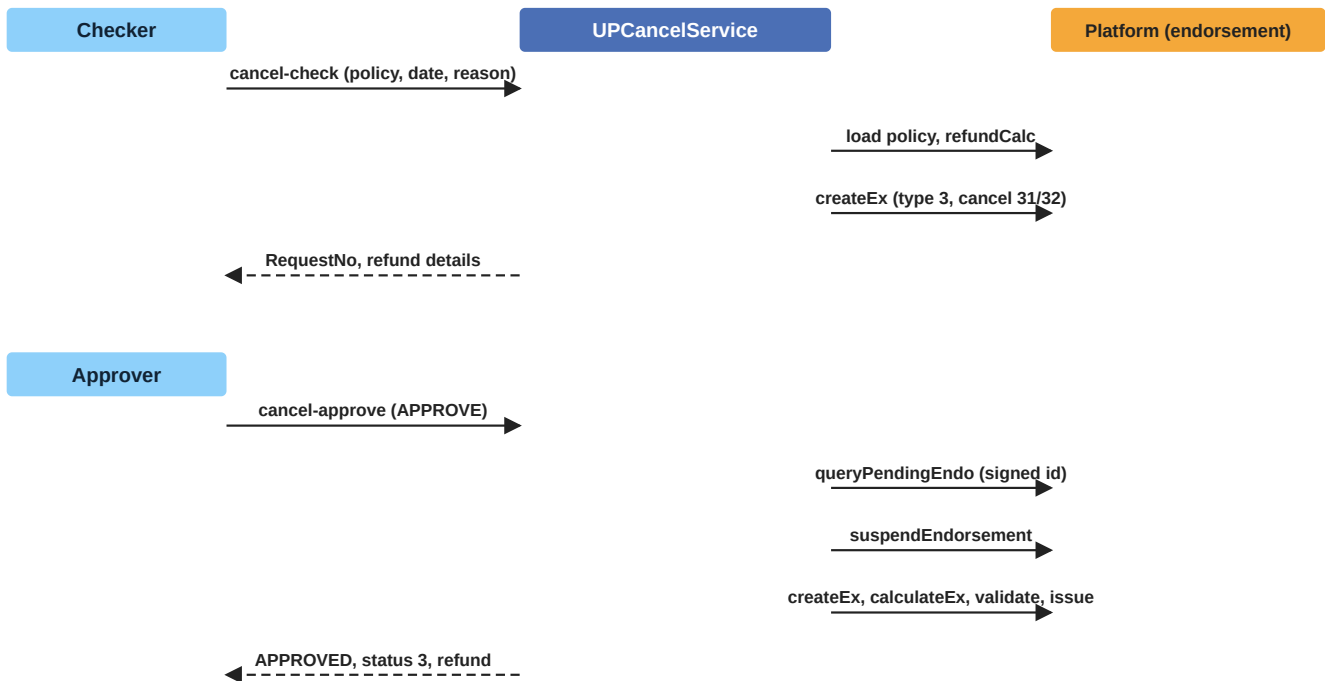
Step	Tool
Tech and market products, tree, DD fields, code tables	<code>build_product.py <spec.json> <step></code> with <code>spec_<CODE>.json</code>
Rating definition per product	<code>rating_build.py <CODE> --confirm</code> (copy of <code>PA001_PREM_CALC</code> bound to the market product), patched by <code>patch_rating.py</code> / <code>patch_rating2.py</code>
Tables and rows	<code>make_tables.py</code> , <code>data_rows.py</code> , <code>uptab.py</code> (create table, save rows), <code>make_rules.py</code> (field rules), <code>stage_*_rows.py</code> (config keys)
Output templates	<code>make_templates2.py</code> builds the bilingual HTML templates from the form designs; <code>gen_all_tpl.py</code> / <code>os_gen.py</code> upload and test them
Cancellation fields	<code>fix_endo_fields.py</code> , <code>bind_cancel_field.py</code> (optional <code>IsAdminChange</code> , not used)
PA001 guard	<code>snapshot.py before after</code>

8. Business Algorithms

8.1 Premium Mode, Term and Age

UPValidator.entryErrors reads the variant's UP_ProductRule row: the premium mode must be in PremiumModes; single premium (SinglePremiumModeCode) needs a term between MinTerm and MaxTerm, other modes need NonSingleTermYears (1); the age at the effective date must be within MinAge / MaxAge and MaxEntryAge, and age + term within MaxExitAge. PremiumModeCode is mandatory where the plans of a variant differ by mode (UPPA001, TL001, CI001, LP001). Premium and commission are never taken from the request: the platform rating calculates them from the plan tables.

8.2 Cancellation (Checker / Approver)



When the platform refuses a step the fresh endorsement stays pending: the answer is HTTP 409 and the request can be approved again or rejected.

Figure 2: Cancellation — Checker and Approver sequence in UPCancelService

The request is the platform's own pending cancellation endorsement; its number is the RequestNo.

Checker — UPCancelService.check

1. resolvePolicy: by PolicyNo or ProposalNo (a proposal must be issued: 409), requireChannel, the product must be enabled (UP_ProductMaster).
2. Policy must be in force (status PolicyStatusInforce); a cancelled policy is 409.
3. pendingCancellation: queries the pending endorsement with the **signed policy id** (id, signature) returned by the platform's own load (ProposalLoadPath), sent once-encoded. Another kind of pending endorsement blocks the request (409). An existing cancellation request is shown again with the date it was created with.
4. refundCalc(policy, date): free look when elapsed days <= FreeLookDays (refund 100%), otherwise premium x unexpired days / policy days x ProRataRefundPercent (rounded MoneyScale, RoundingMode).
5. createRequest: POST EndoBasePath/createEx with EndoType 3, CancelType 31 (free look) or 32 (pro-rata), reason, payment info; for free look the effective date is the policy start.
6. Response: RequestNo, RequestStatus PENDING, PolicyInfo (with insurer), PolicyStatus, PremiumDetails, RefundDetails.

Approver — UPCancelService.decide

1. Same lookup; RequestNo must be the pending endorsement (404 otherwise, also on a second approval).
2. REJECT: suspendEndorsement, answer REJECTED.

3. APPROVE: suspend the saved (thin) request, create a **fresh** endorsement with the same terms (`createEx`), then `calculateEx`, `validate`, `issueEndorsement`. The saved request is replaced because `calculateEx` needs the full `NewPolicy`, which only `createEx` returns.
4. If the platform refuses a step the exception text is capped (`PlatformErrorMaxLength`) and returned as 409
The platform refused the cancellation of <policy>; the request is still pending as <EndoNo>.
5. Success: status 3, `RefundPremium` (same calculation as the Checker), `EndorsementNo`, `RequestStatus` APPROVED.

8.3 Platform Quirks That Shaped the Code

Quirk	Effect / handling
Endorsement service needs the signed policy id (<code>id</code> , <code>signature</code>) and rejects a double-encoded id	Signed id taken from the platform load and sent once-encoded in <code>pendingCancellation</code> .
<code>calculateEx</code> needs the full <code>NewPolicy</code> ; a saved pending endorsement is thin	On APPROVE the saved request is suspended and a fresh <code>createEx</code> is used.
The platform REST client throws its own business exception	<code>UPPlatformClient</code> returns <code>{status, body}</code> ; the approver answers 409 instead of 500.
A policy created without <code>OrgCode</code> gets -1; <code>validate</code> of the endorsement checks <code>OrgCode</code> against code table <code>PubBranch</code> (only value 1)	<code>UPSecurity.defaultOrgCode</code> sets 1 when it is missing; callers send <code>OrgCode</code> 1.
A policy status sent by the caller made the platform fail (500 / 502); a caller policy number was accepted	<code>UPSecurity.stripServerOwned</code> removes <code>PolicyNo</code> , <code>PolicyId</code> , <code>PolicyElementId</code> , <code>ProposalStatus</code> , <code>PolicyStatus</code> .
The cancellation rating reads delta fields that were not bound on the new products	<code>fix_endo_fields.py</code> bound them on the seven tech products (section 7.2).
The platform save drops the calculated plan row	<code>UPUpdateProposal</code> calls <code>persistCalculate</code> after save.
Platform instalment rating re-rounds the premium	Payment info is flattened to <code>Payment*</code> fields and rebuilt in <code>UPShaper.restorePayment</code> .
Responses contain a <code>TempData</code> block with masked / encrypted copies of ID and mobile	Not part of the contract; flagged for review (section 14).
Wrong HTTP method returns 422 from the gateway, not from the code	Documented in the specification.

9. Errors, Logging and Tracing

Situation	Exception thrown in code	HTTP / code
Validation problems	<code>IllegalArgumentException("VALIDATION:" + json)</code> via <code>util.throwValidation</code>	400 UP-VALIDATION with <code>errors[]</code>
Simple bad request	<code>util.fail(msg) -> IllegalArgumentException</code>	400 UP-400
State conflict	<code>IllegalArgumentException("CONFLICT:...")</code>	409 UP-409
Bad number	<code>NumberFormatException</code>	400 UP-400, text <code>MsgInvalidNumber</code>
Missing configuration / table row	<code>IllegalStateException</code>	500 UP-CONFIG, text <code>MsgConfigError</code> ; detail in the log
Platform business error	<code>ApiException</code> (message sanitised by <code>isInternalText</code>)	status of the platform, its code; internal text replaced by <code>MsgServerError</code>
Not found	<code>errorHandler.notFound(api, msg)</code>	404 UP-404
Anything else	<code>Exception</code>	500 UP-500, generic text; full exception logged with the trace id

Tracing: every error body carries `trace_id`. Find the call in the platform log with `imo log query` using that trace id (application logs of the iComposer runtime); UP-CONFIG messages name the missing key only in the log. Do not add `println` or secrets to the code; use the slf4j logger.

10. How-To Recipes

Add or change a validation rule

1. Read the rows of `UP_FieldRule` (`imo config datatable record-list --id 1188575333`).
2. Add a row (`record-batch-save`, a new unique Id) or change one (`record-save` with its `RecordId`): `Api`, `Path`, `Label`, `Required`, `DataType`, `limits`, `Pattern`, `CodeTable`.
3. Test the API with a valid and an invalid value; no deployment is needed.
4. Update the specification (`api_spec_data.py` reads the rules) and the policy model.

Rule that needs code (cross-field, product rule)

1. Edit `UPValidator.entryErrors / validate` (keep one `util.fieldError(path, code, message)` per problem).
2. Push `UPValidator`; push nothing else unless a signature changed.
3. Add a test case (`run_tests.py`, business cases) and a line in the specification.

Change free look days or refund percent

1. Edit `FreeLookDays / ProRataRefundPercent` of the variant row in `UP_ProductRule` (`record-save`).
2. Check with `cancel-check` on a test policy; update the specification section 6.2 if the value differs from the documented 15 / 80.

Add a plan or change a premium

1. Add or change rows in `UP_ProductPlanRelation` and `UP_PlanBenefitRelation`; add the plan id to the `Plan` code table if it is new.
2. Run `Get Plans` and `Create Proposal` for the plan and compare the premium with the table (the rating reads the tables).

Add a product variant (sub-product)

1. Add `UP_ProductMaster`, `UP_ProductRule`, plan and cover rows, `UP_DocTemplate` rows and the output templates (`make_templates2.py`).
2. Add test samples (`samples/req/<TAG>_proposal.json`) so `ref_flow.py` can run the variant.

Add a new product code

1. Create the tech and market product, the DD tree and bindings and the rating definition with `build_product.py` and `rating_build.py` (section 7.3).
2. Bind the cancellation delta fields with `fix_endo_fields.py <TAG>`.
3. Add the table rows as for a variant; add the code to the product lists of the tooling (`products.json`, test scripts).

Add a channel

1. Add a row to `UP_Channel` (`ChannelCode`, `Description`, `IsActive Y`). The `ChannelCode` rule checks the value against this table.

Add or change the insurer

1. Add a row to `UP_Carrier`; set `UP_ProductMaster.CarrierCode` of the variants to it. Responses show the name from the table; a product whose carrier row is missing or inactive returns `UP-CONFIG`.

Add a master (code) table to Master Data

1. Append the table name to the value of `UP_ApiConfig.MasterTables` (`record-save` of the row with its `RecordId`).

Add a field to a response

1. Policy-level: set it in `UPShaper.shapeResponse` (or the API for `Get Plans`, which does not use the shaper).
Cancellation: `UPCancelService.policyInfo` and the `out` maps. Commission: `UPDocBuilder.commissionOfPolicy`. Feed: add a `UP_FeedColumn` row.
2. Add the field to the specification (`api_spec_data.py` reads recorded responses: re-run `ref_flow.py` for one plan per product).

Add a new API

1. `imo icomposer create api ...` (path `/up/<name>`), copy the pattern of a small API (readInput, validate, call, shape, errorHandler).
2. Add `UP_FieldRule` rows for the new API name, push, add cases to `run_tests.py` and `security_probe.py`, document in the specification.

Change a message text or format

1. Edit the `UP_ApiConfig` value (`MsgServerError`, `MsgConfigError`, `MsgInvalidNumber`, `DateFormatDoc`, `MoneyFormat`, ...).

Move to another environment

1. Change `AllowedHosts`, `GatewayUrl`, `PrintUrl` (section 13) and re-check the ids listed in the deployment plan.

11. Test Tooling

All scripts live in `uponly-build` and read the token and gateway from `env.sh` (`source env.sh` sets `B`, `GW`, `TOK`; the token is fetched with `imo auth prepare --profile portal:uponly`). Never commit tokens.

Script	What it does
<code>carrier_check.py</code>	One flow per product (first plan): CarrierCode / CarrierName in every API, wrong CarrierCode, master data and feed file; output <code>carrier/results.json</code> .
<code>ref_flow.py</code>	Runs the 13-step flow (get plans ... cancel approve, load cancelled policy) for every plan of a variant; <code>ONLY_PLAN=1</code> limits it to one plan; output in <code>samples/ref/<TAG>_P<plan>/</code> .
<code>run_tests.py</code>	Generates and runs the API test suite (1,085 cases: field validation, business rules, state transitions, cancellation incl. refund checks per product, master data, feed file, robustness). Results in <code>tests/results/</code> and <code>tests/results.json</code> .
<code>security_probe.py</code>	Live security probe (token tampering, channel isolation, injection, tampering, resource abuse, headers, errors, request hygiene); output <code>security/results.json</code> .
<code>snapshot.py</code>	Read-only snapshot of PA001 related objects; compare <code>before</code> and <code>after</code> to prove PA001 is unchanged.

Collections: `make_sohar_collection.py` (single Sohar Bank collection, one flow per product), `make_plan_collections.py` (one collection per plan with recorded failure examples), `make_cases_by_product.py`. Reports: `gen_report.py`, `make_excel.py`. Documents: `build_policy_model.py` (workbook), `spec_content.py` + `render_spec.py` / `render_docx.py` (specification), `dev_content.py` (this guide).

All scripts in the build folder

Script	Purpose (first docstring line)
<code>api_spec_data.py</code>	One structured specification per API (inputs with M / O / C, outputs with A / C), shared by the workbook and the specification document. Inputs come f
<code>bind_cancel_field.py</code>	Adds ONE field binding, <code>IsAdminChange</code> (string), to the cancellation endorsement object (id 300224988) for the Uponly TECH products only. It never touc
<code>build_api_docs.py</code>	External API documentation (Markdown) + OpenAPI 3.0 spec for the Uponly insurance APIs. Internal details (tenant tables, configuration keys, code name
<code>build_policy_model.py</code>	Builds <code>Afillar_PolicyModel_v0.4.xlsx</code> in the layout of the PA policy model workbook (v0.1), extended to all Uponly products and APIs.
<code>build_product.py</code>	Component-path fields are created as temporary (<code>IsTemp=Y</code> , not stored). Make them persisted like Uponly's.
<code>carrier_check.py</code>	One flow per product (first variant, first plan): CarrierCode / CarrierName must be returned by every API. Writes <code>carrier/results.json</code> .
<code>carrier_setup.py</code>	Creates the insurer table <code>UP_Carrier</code> and the rows that expose CarrierCode / CarrierName (field rules, feed columns, master table list). Idempotent: id
<code>code_facts.py</code>	Facts extracted from the real source files, tables and scripts, so that the developer guide cannot drift from the code.
<code>dev_content.py</code>	Developer guide content (block list). Reference sections are generated from the real source, tables and scripts.
<code>fix_endo_fields.py</code>	Admin script: makes the platform's cancellation calculation pass for an Uponly product by binding the fields it reports as 'not bound'. usage: <code>fix_end</code>
<code>make_plan_collections.py</code>	One Postman collection per plan; one folder per API; real flow request/response + recorded failure examples.
<code>make_ref_postman.py</code>	Builds the TrialX collection in exactly the structure of the UPONLY_UAT reference collection.
<code>make_rules.py</code>	Creates <code>UP_FieldRule</code> , <code>UP_ApiConfig</code> , <code>UP_DocWording</code> , <code>UP_BenefitText</code> in Uponly and loads their rows.
<code>make_sohar_collection.py</code>	Single 'Sohar Bank' collection: one example per product, token mapped, values parameterized.
<code>make_tables.py</code>	Create the <code>UP_ data</code> tables in Uponly (definitions and rows copied from the tested TrialX tables).
<code>make_templates2.py</code>	Bilingual (english, arabic) paragraphs from the long text rows of the application docx.

Script	Purpose (first docstring line)
md_to_docx.py	Minimal Markdown -> docx (headings, paragraphs, bullets, tables, bold, code spans). usage: md_to_docx.py in.md out.docx
model_data.py	Collects the facts for the policy model workbook and the external API documentation from the build folder (product tables, field rules, code tables, s
os_gen.py	usage: os_gen.py TEMPLATE_NAME html_file data.json out.pdf (uploads and generates)
rating_build.py	Create <CODE>_PREM_CALC (copy of Uponly PA001_PREM_CALC) bound to a market product in the current tenant. usage: rating_build.py <PRODUCTCODE> [--conf
ref_flow.py	Runs the PA001-style 11 step flow for every plan of every variant, using the request bodies of the UPONLY_UAT reference collection as the template, an
render_docx.py	DOCX version of the specification (same content blocks): real heading styles, TOC field, repeating table header rows, page numbers.
render_spec.py	Renders the specification content to PDF (HTML + Chromium, TOC page numbers from a first pass) and DOCX (python-docx), in the layout of the InsureMO r
run_tests.py	Stored copy: large responses are cut to keep the result files small (the live copy keeps the full body for the follow-up steps).
security_probe.py	Live security (VAPT style) probe of the deployed Uponly APIs in the test tenant. Writes security/results.json. Each check: id, category, title, what w
snapshot.py	Read-only snapshot of PA001 related Uponly objects. usage: snapshot.py before after
spec_content.py	Content model of the API specification (list of blocks) in the structure of the InsureMO reference specification.
split_util.py	Splits UPDataUtil into focused functions and rewrites the callers. Usage: split_util.py <src-root> <out-root>
stage_a_rows.py	Stage A: new configuration rows (no defaults in code) + UP_FeedColumn.
stage_cd_rows.py	Stages C + D: ChannelCode rules on every API, cancel-check / cancel-approve rules and config keys, ChannelCode feed column.
status_cancel_spike.py	Spike: can an issued Uponly policy be set to Cancelled through the platform's own proposal update calls? Works on ONE throwaway Uponly policy only (th
uat_content.py	UAT deployment plan content (block list) + data for the checklist workbook.
uptab.py	Small helpers to create / extend Uponly data tables (same approach as make_rules.py).

12. Test Procedure for a Change

1. Before: `snapshot.py before` (PA001 guard) and copy `ic` to a backup folder.
2. Make the change; push with `imo icomposer push current` (the compile check must say `Compiled: yes`).
3. Run the affected cases (`run_tests.py all <filter>`) and, for response changes, `ref_flow.py` for one plan per product and `carrier_check.py`.
4. For anything touching input handling or errors run `security_probe.py`.
5. After: `snapshot.py after` and compare with `before` (PA001 and PA001_TECH hashes identical).
6. Update the documents (specification, policy model, this guide) and the collection (`make_sohar_collection.py`), then run the collection in Newman.
7. Rollback: push the saved files back; for tables restore the saved row.

13. Environments and Deployment

The complete inventory, ordered steps, verification and rollback for deploying to UAT are in the separate document *Afillar API UAT Deployment Plan*. In short: 10 functions and 12 APIs, 12 `UP_` tables with their rows, the product configuration on the platform, the output templates and the environment settings. The keys that differ per environment are `AllowedHosts`, `GatewayUrl` and `PrintUrl`; the ids of products, data tables and templates are tenant specific and must be re-resolved, not copied.

14. Known Limitations and Backlog

Item	Description	Suggested action
Role / branch access	Any authenticated user of a channel can call any API; the ChannelCode check isolates channels only.	Access table per API and role (deferred by decision).
Security baselines in code	UPSecurity constants with optional override keys.	Add the override keys / UP_FieldRule patterns and make a missing key UP-CONFIG.
Gateway findings	Wildcard CORS, no rate limiting, OPTIONS 500, attack-like query strings hold the connection.	Platform team.
ID number and TempData in responses	The ID number is returned in clear and a TempData block with encrypted copies is part of the platform body.	Decide on masking; strip TempData in UPShaper.
Idempotency and audit	No idempotency key; no audit record of who approved a cancellation beyond the platform endorsement.	Idempotency key on create / approve; audit table.
Unit tests	Only black-box API tests exist.	Data-driven tests for pure functions (refundCalc, entryErrors, scan).
Per-plan collections	52 per-plan collections were not regenerated after the insurer fields were added (still valid; examples lack the two fields).	Re-run make_plan_collections.py after ref_flow.py for all plans.
PDF templates	The insurer name is not printed on the documents.	Add CarrierName to the print data and the templates.

14.1 Troubleshooting

Symptom	Likely cause	Check
HTTP 500 UP-CONFIG	A configuration key, table row (carrier, channel, product) is missing	Log of the trace id names the key / table
Cancel approve returns 409 platform refused	Platform validation of the endorsement (for example OrgCode -1 on an old policy)	Text after the policy number; run probe_approve.py <PolicyNo> to see each step
Get Plans returns no plan	No active plan row for the variant and premium mode	UP_ProductPlanRelation
400 UNKNOWN_VALUE on a code	Value missing in the code table	imo config codetable data --name X
Push fails to compile	Callee not pushed yet or a Groovy static type error	Push order (section 3.4)
401 from every call	Token expired	Get a new token
Document not generated	Template missing or print host not in AllowedHosts	UP_DocTemplate, PrintUrl, AllowedHosts

14.2 First-Day Checklist for a New Developer

1. Install imo, log in to the MC tenant (imo auth login --profile portal:uponly).
2. Clone the source export, read sections 2 and 4 of this guide.
3. source env.sh, run carrier_check.py (one flow per product) to see the APIs work.
4. Open UPCancelService.groovy and UPValidator.groovy next to sections 8 and 6.3.
5. Make a harmless change in a copy tenant or a branch: add a UP_FieldRule row and see the 400.
6. Read the specification and the policy model workbook (API-Field sheets) for the contract.

Appendix A. File Inventory

File (under ic/)	Lines	Status
src/dev/Tenant/UP_PRODUCTS_API/api/UPApplicationDocGeneration/UPApplicationDocGeneration.groovy	24	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelApprove/UPCancelApprove.groovy	24	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelCheck/UPCancelCheck.groovy	25	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPCancelPolicy/UPCancelPolicy.groovy	5	inactive
src/dev/Tenant/UP_PRODUCTS_API/api/UPCommissionQuery/UPCommissionQuery.groovy	136	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPDocGeneration/UPDocGeneration.groovy	23	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPFeedFile/UPFeedFile.groovy	178	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPGetPlans/UPGetPlans.groovy	104	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPIssue/UPIssue.groovy	51	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPListMasterTable/UPListMasterTable.groovy	37	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPLoadPolicy/UPLoadPolicy.groovy	36	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPProposal/UPProposal.groovy	37	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPProposalReject/UPProposalReject.groovy	43	active
src/dev/Tenant/UP_PRODUCTS_API/api/UPUpdateProposal/UPUpdateProposal.groovy	58	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPCancelService/UPCancelService.groovy	303	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDataUtil/UPDataUtil.groovy	235	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDocBuilder/UPDocBuilder.groovy	195	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPDocService/UPDocService.groovy	105	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPErrorHandler/UPErrorHandler.groovy	100	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPPlatformClient/UPPlatformClient.groovy	49	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPProductData/UPProductData.groovy	202	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPSecurity/UPSecurity.groovy	116	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPShaper/UPShaper.groovy	126	active
src/dev/Tenant/UP_PRODUCTS_API/function/UPValidator/UPValidator.groovy	334	active

Appendix B. Table Definitions

UP_ApiConfig (id 1188575596, 81 rows)

Column	Data type	Primary key
Id	-4	Y
Api	-4	
Scope	-4	
Key	-4	
Value	-4	

UP_FieldRule (id 1188575333, 138 rows)

Column	Data type	Primary key
Id	-4	Y
Api	-4	
Path	-4	
Label	-4	
Required	-4	
DataType	-4	
MinLength	-2	
MaxLength	-2	
Pattern	-4	
MinValue	-3	
MaxValue	-3	
CodeTable	-4	
CodeColumn	-4	
EnumValues	-4	
AppliesTo	-4	
WhenPath	-4	
WhenValue	-4	

UP_ProductMaster (id 1188475672, 11 rows)

Column	Data type	Primary key
Id	-4	Y
Description	-4	
LOB	-4	
IMOProductCode	-4	
IsRenewable	-4	
IsActive	-4	
IsFlatRate	-4	
Premium	-3	
SumInsured	-3	
HasPlans	-4	
CarrierCode	-4	
CarrierProductCode	-4	
CarrierProductName	-4	

Column	Data type	Primary key
IsResident	-4	
BenefitRelationTableName	-4	
LobId	-2	
CarrierName	-4	

UP_ProductPlanRelation (id 1188475676, 57 rows)

Column	Data type	Primary key
Id	-4	Y
IMOProductCode	-4	
IMOPlanCode	-4	
IMOPlanName	-4	
CarrierCode	-4	
CarrierProductCode	-4	
CarrierProductName	-4	
CarrierPlanCode	-4	
CarrierPlanName	-4	
IsActive	-4	
Premium	-3	
SumInsured	-3	
Commission	-3	
PremiumModeCode	-4	

UP_PlanBenefitRelation (id 1188474890, 148 rows)

Column	Data type	Primary key
Id	-4	Y
IMOProductCode	-4	
IMOPlanCode	-4	
CarrierCode	-4	
CarrierProductCode	-4	
CarrierPlanCode	-4	
BenefitCode	-4	
BenefitDesc	-4	
BenefitLevel	-4	
ParentBenefitCode	-4	
BenefitLimit	-4	
BenefitLimitDesc	-4	
IsMainBenefit	-4	
IsAdditionalBenefit	-4	
Notes	-4	
IsApplicable	-4	
LimitBasis	-4	
MaxDays	-2	

UP_ProductRule (id 1188475684, 11 rows)

Column	Data type	Primary key
Id	-4	Y
ProductCode	-4	
ProductSubcode	-4	
PremiumModes	-4	
MinTerm	-2	
MaxTerm	-2	
MinAge	-2	
MaxAge	-2	
MaxEntryAge	-2	
MaxExitAge	-2	
MandatoryDocuments	-4	
SinglePremiumRule	-4	
FreeLookDays	-2	
ProRataRefundPercent	-3	

UP_DocTemplate (id 1188475689, 22 rows)

Column	Data type	Primary key
Id	-4	Y
DocumentCode	-4	
TemplateName	-4	
FileName	-4	
ProductCode	-4	
ProductSubcode	-4	
DocType	-4	

UP_DocWording (id 1188556760, 27 rows)

Column	Data type	Primary key
Id	-4	Y
ModeCode	-4	
Key	-4	
Value	-4	

UP_BenefitText (id 1188575620, 21 rows)

Column	Data type	Primary key
Id	-4	Y
ProductCode	-4	
BenefitCode	-4	
NameEn	-4	
NameAr	-4	

UP_FeedColumn (id 1188591810, 29 rows)

Column	Data type	Primary key
Id	-4	Y
Sequence	-2	

Column	Data type	Primary key
Heading	-4	
Source	-4	

UP_Channel (id 1188591925, 1 rows)

Column	Data type	Primary key
Id	-4	Y
ChannelCode	-4	
Description	-4	
IsActive	-4	

UP_Carrier (id 1188663015, 1 rows)

Column	Data type	Primary key
CarrierCode	-4	Y
CarrierName	-4	
IsActive	-4	